

Reviewer Pack — Minimal Order of Modular Forms and Resolution Proof Width Co...

Entry 77803c2e0491 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 10:18:58 UTC

Minimal Order of Modular Forms and Resolution Proof Width Correlation

Entry ID: 77803c2e0491 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 10:18:58 UTC

1. Conjecture

Field A (mathematical branch): Modular Form Theory **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every Tseitin formula φ_G associated with a d -regular graph G , the minimal order of elliptic modular forms ($m_order(\varphi_G)$) corresponding to the graph's eigenvalues is linearly correlated with its resolution proof width $w(\varphi_G)$, such that $m_order(\varphi_G) = \Theta(w(\varphi_G))$.

Rationale (proposer's reasoning):

Modular form theory provides a rich algebraic structure that could potentially encode non-trivial information about the complexity of certain computational problems, such as resolution proofs. The minimal order of modular forms might reflect the complexity inherent in the structure of the graph represented by the Tseitin formula, providing a novel approach to complexity lower bounds.

Taxonomy category: `minimal_modular_form_order` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9ad175a380136a42

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given d -regular graph G with n vertices, we consider it supported if the Pearson correlation coefficient (r) between $m_order(\varphi_G)$ and $w(\varphi_G)$ is ≥ 0.8 AND $r^2 \geq 0.9$, where $m_order(\varphi_G)$ is the minimal order of elliptic modular forms and $w(\varphi_G)$ is the resolution proof width. It is considered falsified if any seed produces a correlation coefficient $r < 0.5$ or if any seed results in an absolute difference $|r^2 - 1| > 0.1$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal order of elliptic modular forms" AND "resolution proof complexity" - "Tseitin formula" AND "elliptic modular form" AND "proof width" - "graph eigenvalues" AND "modular form theory" AND "resolution proof complexity"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return A

def determinant(A):
    n = len(A)
    det = 1
    for i in range(n):
        for j in range(i+1, n):
            if A[i][i] == 0:
                return 0
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    det *= A[i][i]
    return det
```

```

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(d, n):
        if d * n % 2 != 0:
            return None
        G = [[0] * n for _ in range(n)]
        edges = set()
        while len(edges) < (d * n) // 2:
            u = random.randint(0, n-1)
            v = random.randint(0, n-1)
            if u != v and (u, v) not in edges and (v, u) not in edges:
                G[u][v] = 1
                G[v][u] = 1
                edges.add((u, v))
        return G

    def tseitin_formula(G):
        n = len(G)
        literals = [f"x{i}" for i in range(n)]
        clauses = []
        for i in range(n):
            clause = [literals[i]]
            for j in range(n):
                if G[i][j]:
                    clause.append(f"~{literals[j]}")
            clauses.append(clause)
            for j in range(i+1, n):
                for k in range(j+1, n):
                    if G[i][k] and G[j][k]:
                        clauses.append([f"~{literals[i]}", f"~{literals[j]}", literals[k]])
        return clauses

    def eigenvalues(G):
        n = len(G)
        A = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                if G[i][j]:
                    A[i][j] = 1
        det_A = determinant(A)
        if det_A == 0:
            return None
        eigenvals = []
        for k in range(1, n+1):
            I = [[0] * n for _ in range(n)]
            for i in range(n):
                I[i][i] = 1
            A_k = [[A[i][j]**k for j in range(n)] for i in range(n)]
            det_A_k = determinant(A_k)
            if det_A_k == 0:
                eigenvals.append(k)
        return eigenvals

    def m_order(eigenvals):
        if not eigenvals:

```

```

        return None
    min_order = float('inf')
    for val in eigenvals:
        order = sum(1 for e in eigenvals if abs(e - val) < 1e-6)
        if order < min_order:
            min_order = order
    return min_order

def resolution_proof_width(clauses):
    n = len(clauses)
    width = max(len(c) for c in clauses)
    return width

d = random.randint(2, 3)
n = random.randint(5, 40)
G = generate_d_regular_graph(d, n)
if not G:
    return {
        "metric_name": "m_order",
        "metric_value": None,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "graph_not_d_regular"
    }

clauses = tseitin_formula(G)
eigenvals = eigenvalues(G)
if not eigenvals:
    return {
        "metric_name": "m_order",
        "metric_value": None,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "eigenvalue_computation_failed"
    }

m = m_order(eigenvals)
w = resolution_proof_width(clauses)

return {
    "metric_name": "m_order",
    "metric_value": m,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": False if m is None or w is None else True,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43]
    results = []

    for seed in seeds:

```

```

    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

if all(r["conjecture_holds"] for r in results):
    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = 1.0
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    counterexample = "m_order_computation_failed"
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

unterexample': 'eigenvalue_computation_failed'}
TRIAL: {'metric_name': 'm_order', 'metric_value': None, 'instances_tested': 1, 'n_max': 35, 'conjecture_holds': False}
TRIAL: {'metric_name': 'm_order', 'metric_value': None, 'instances_tested': 1, 'n_max': 14, 'conjecture_holds': False}
TRIAL: {'metric_name': 'm_order', 'metric_value': None, 'instances_tested': 1, 'n_max': 9, 'conjecture_holds': False}
TRIAL: {'metric_name': 'm_order', 'metric_value': None, 'instances_tested': 1, 'n_max': 28, 'conjecture_holds': False}
TRIAL: {'metric_name': 'm_order', 'metric_value': None, 'instances_tested': 1, 'n_max': 26, 'conjecture_holds': False}
TRIAL: {'metric_name': 'm_order', 'metric_value': None, 'instances_tested': 1, 'n_max': 9, 'conjecture_holds': False}
TRIAL: {'metric_name': 'm_order', 'metric_value': None, 'instances_tested': 1, 'n_max': 38, 'conjecture_holds': False}
TRIAL: {'metric_name': 'm_order', 'metric_value': None, 'instances_tested': 1, 'n_max': 36, 'conjecture_holds': False}
TRIAL: {'metric_name': 'm_order', 'metric_value': None, 'instances_tested': 1, 'n_max': 24, 'conjecture_holds': False}
TRIAL: {'metric_name': 'm_order', 'metric_value': None, 'instances_tested': 1, 'n_max': 32, 'conjecture_holds': False}
TRIAL: {'metric_name': 'm_order', 'metric_value': None, 'instances_tested': 1, 'n_max': 38, 'conjecture_holds': False}
RESULT: FALSIFIED counterexample="m_order_computation_failed" first_failing_seed=11

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code fails to compute eigenvalues correctly, as indicated by the 'eigenvalue_computation_failed' counterexample. This suggests that the metric definition bug is present, as the test does not actually measure what the statement says.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test code fails to compute eigenvalues correctly, indicating a potential bug in the metric definition. The pre-registered support condition was no | next: Investigate and correct the eigenvalue computation issue. Once resolved, retest the conjecture with a corrected implementation.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16925
2	propose	ollama_remote	glm4:latest	0	0	14244
3	preregistration	ollama_remote	glm4:latest	0	0	15346
4	novelty	ollama_remote	glm4:latest	0	0	8442
5	novelty	ollama_remote	glm4:latest	0	0	12291
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	28441
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25336
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	32735
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17577
10	critic	ollama_remote	glm4:latest	0	0	10699
11	judge	ollama_remote	glm4:latest	0	0	12491

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 194526 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/77803c2e0491.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/77803c2e0491.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/77803c2e0491.tar.gz` (if generated)